

Titles

Project Name – To-Do List Application

Build by – Shashikant Sahu

Batch – 1st March 2025

Table of Contents

1. Introduction
2. Organized File Structure
3. Creativity and Presentation
 - Technologies Used
 - code
 - Screenshot
4. GitHub Link

Introduction

The **To-Do List Application** is a simple and interactive task management tool built using React and Vite. It allows users to keep track of their daily tasks by adding, editing, marking as complete, and deleting them. The application uses `localStorage` to save tasks so they persist even after refreshing or closing the browser.

This project is designed using functional components, React hooks (`useState`, `useEffect`), and follows clean code practices with modular components like `App`, `Header`, `ToDoList`, and `ToDoItem`.

It is a great beginner-friendly project for understanding React fundamentals, state management, event handling, and component-based UI development.

Organized File Structure

Creating a well-organized file structure is crucial for readability, maintainability, and collaboration in any web project. My project is already structured well, and here's a breakdown of each component:

Description of Files:

◆ index.html

- The main HTML file used by Vite.
 - It contains a `<div id="root"></div>` where your React app is injected.
 - You typically don't edit this much.
-

◆ main.jsx

- Entry point of your React app.
- This is where the App component is rendered into the root element.

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
import App from './App';
```

```
import './App.css';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

◆ App.jsx

- The main component that manages:
 - App-wide state (tasks)
 - Logic for add, delete, edit, complete
 - Communication with localStorage
 - It renders the Header and ToDoList components and passes them necessary props.
-

◆ App.css

- Contains the custom styling for your app.
 - Styles the layout, buttons, input, list items, and overall UI.
-

◆ components/Header.jsx

- Displays the application title and input field for adding new tasks.
 - Handles form input using its own local state.
 - Calls a function passed via props (addTodo) to add tasks to the main list.
-

◆ components/ToDoList.jsx

- Receives the tasks array via props.
 - Uses .map() to render each task using the ToDoItem component.
 - Passes down onComplete, onDelete, and onEdit handlers.
-

◆ components/ToDoItem.jsx

- Represents a single to-do item.
 - Supports:
 - Marking task as complete/incomplete
 - Editing task text
 - Deleting task
 - Uses props to trigger changes in the parent (App).
-

◆ package.json

- Lists project dependencies like react, vite, and react-icons.
 - Also contains scripts like npm run dev to start the app.
-

◆ vite.config.js

- Configuration file for Vite.
 - Usually left unchanged unless you need to customize the dev server, alias paths, etc.
-

◆ README.md

- Documentation file explaining:
 - What the app does
 - How to install and run it
 - Features
 - Technologies used
- .

Creativity and Presentation

This To-Do List application demonstrates a clean, intuitive user interface along with thoughtful design choices to ensure ease of use and clarity. Each task is visually distinct, and users can easily identify and interact with buttons for editing, deleting, and completing tasks. Responsive feedback is provided through icons and input validation alerts, enhancing the overall user experience.

1. Technologies Used

Technology	Purpose
React	Core library for building the user interface using components
Vite	Fast development server and build tool for modern React apps
JavaScript (ES6+)	Main language for logic and interaction
React Hooks (useState, useEffect)	To manage component state and side effects like localStorage
LocalStorage API	For saving tasks so they persist after refreshing the page
CSS	For custom styling and layout
React Icons	For user-friendly icons on buttons (edit, delete, complete, undo)

2. Code

index.html code :-

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>To-Do React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

main.jsx code :-

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

App.jsx Code:-

```
import './App.css';
import { useEffect, useState } from "react";
import Header from "./components/Header";
import ToDoList from "./components/ToDoList";

function App() {
  const [tasks, setTasks] = useState(() => {
```



```

//load tasks from localStorage
const savedTasks = localStorage.getItem("tasks");
return savedTasks ? JSON.parse(savedTasks) : [];
});

const [newTask, setNewTask] = useState("");

//save to localStorage whenever tasks change
useEffect(() => {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}, [tasks]);

function addTask() {
  if (newTask === "") {
    alert("List cannot be empty !");
    return;
  }
  if (newTask.length < 5) {
    alert("List text should be at least 5 characters long !");
    return;
  }

  if (newTask.trim()) {
    const newItem = {
      id: Date.now(),
      text: newTask,
      completed: false,
    };
    setTasks([...tasks, newItem]);
    setNewTask("");
  }
  // console.log(tasks);
}

function handleDeleteTask(id) {
  // Logic to handle task deletion
  setTasks(tasks.filter((task) => task.id !== id));
}

```

```

function handleCompleteTask(id) {
  // Logic to handle task completion
  setTasks(
    tasks.map((task) =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  );
}

```

```

function handleEditTask(id, updatedText) {
  setTasks(
    tasks.map((task) =>
      task.id === id ? { ...task, text: updatedText } : task
    )
  );
}

```

```

return (
  <>
  <Header />
  <div className="todo-add">
    <input
      value={newTask}
      onChange={(e) => setNewTask(e.target.value)}
      className="todo-input"
      type="text"
      placeholder="Write Your text here..."
    />
    <button onClick={addTask} className="add-button">
      Add
    </button>
  </div>
  <ToDoList
    tasks={tasks}
    onComplete={handleCompleteTask}
    onDelete={handleDeleteTask}
    onEdit={handleEditTask}
  />
)

```

```
    />  
  </>  
);  
}
```

```
export default App;
```

App.css Code:-

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
  font-family: Arial, Helvetica, sans-serif;  
}  
/* Header.jsx css */  
.heading {  
  font-size: 2rem;  
  text-align: center;  
  margin: 20px 0;  
}  
/* App.jsx css */  
.todo-add {  
  display: flex;  
  justify-content: center;  
  padding: 20px;  
  gap: 5px;  
}  
.todo-input {  
  width: 60%;  
  padding: 10px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
}  
.add-button {  
  padding: 10px 45px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
  font-weight: bold;
```

```
}  
.add-button:hover {  
  background-color: #969292;  
}
```

```
/* ToDoItem.jsx css */
```

```
.item {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  padding: 10px;
```

```
}  
.item-text {  
  font-size: 1rem;  
  text-align: justify;  
  width: 60%;  
  padding: 10px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
}
```

```
.item-complete {  
  font-size: 1.3rem;  
  padding: 6px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
  margin: 2px;  
}
```

```
.item-edit {  
  font-size: 1.3rem;  
  padding: 6px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
  margin: 2px;  
}
```

```
.item-delete {  
  font-size: 1.3rem;
```

```

padding: 6px;
border-radius: 5px;
border: 1px solid #ccc;
margin: 2px;
}
.item-complete:hover {
background-color: #969292;
color: green;
}
.item-edit:hover {
background-color: #969292;
color: blue;
}
.item-delete:hover {
background-color: #969292;
color: red;
}

```

Components/Header.jsx Code:-

```

import "../App.css";

function Header() {
  return (
    <>
      <h1 className="heading">ToDo List</h1>
    </>
  );
}
export default Header;

```

Components/ToDoList.jsx Code:-

```

import ToDoItem from "../ToDoItem";

function ToDoList(props) {
  return (
    <>
      <ul className="todo-list">
        {props.tasks.map((task) => (

```

```

      <ToDoItem
        key={task.id}
        task={task}
        onCompleted={props.onComplete}
        onDeleted={props.onDelete}
        onEdited={props.onEdit}
      />
    )}
  </ul>
</>
);
}
export default ToDoList;

```

Components/ToDoItem.jsx Code:-

```

import { useState } from "react";
import { MdDeleteForever } from "react-icons/md";
import { MdEditSquare } from "react-icons/md";
import { FaUndo } from "react-icons/fa";
import { MdFileDownloadDone } from "react-icons/md";

function ToDoItem(props) {
  const [isEditing, setIsEditing] = useState(false);
  const [editedText, setEditedText] = useState(props.task.text);

  function handleEditSubmit() {
    props.onEdited(props.task.id, editedText);
    setIsEditing(false);
  }

  return (
    <li className="todo-item"
      style={{ textDecoration: props.task.completed ? "line-through" : "none" }}
    >
      {isEditing ? (
        <div className="item">
          <input
            className="todo-input"

```

```

      type="text"
      value={editedText}
      onChange={(e) => setEditedText(e.target.value)}
    />
    <button className="add-button" onClick={handleEditSubmit}>
      Save
    </button>
  </div>
) : (
  <div className="item">
    <p className="item-text">{props.task.text}</p>
    { /* Show Complete/Undo button */ }
    <button
      className="item-complete"
      onClick={() => props.onCompleted(props.task.id)}
    >
      {props.task.completed ? <FaUndo /> : <MdFileDownloadDone />}
    </button>
    { /* Conditionally render the Edit button */ }
    { !props.task.completed && (
      <button className="item-edit" onClick={() => setIsEditing(true)}>
        <MdEditSquare />
      </button>
    ) }
    { /* Show Delete button */ }
    <button
      className="item-delete"
      onClick={() => props.onDeleted(props.task.id)}
    >
      <MdDeleteForever />
    </button>
  </div>
) }
</li>
);
}
export default ToDoItem;

```

3. Screenshot :-

ToDo List

Add

Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph. A paragraph is defined as "a group of sentences or a single sentence that forms a unit" (Lunsford and Connors 116). Length and appearance do not determine whether a section in a paper is a paragraph. For instance, in some styles of writing, particularly journalistic styles, a paragraph can be just one sentence long. Ultimately, a paragraph is a sentence or group of sentences that support one main idea. In this handout, we will refer to this as the "controlling idea," because it controls what happens in the rest of the paragraph.



Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph.



Paragraphs are the building blocks of papers.



localhost:5173 says
List cannot be empty !

OK

Add

Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph. A paragraph is defined as "a group of sentences or a single sentence that forms a unit" (Lunsford and Connors 116). Length and appearance do not determine whether a section in a paper is a paragraph. For instance, in some styles of writing, particularly journalistic styles, a paragraph can be just one sentence long. Ultimately, a paragraph is a sentence or group of sentences that support one main idea. In this handout, we will refer to this as the "controlling idea," because it controls what happens in the rest of the paragraph.



Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph.



Paragraphs are the building blocks of papers.



localhost:5173 says

List text should be at least 5 characters long !

OK

sd

Add

Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph. A paragraph is defined as "a group of sentences or a single sentence that forms a unit" (Lunsford and Connors 116). Length and appearance do not determine whether a section in a paper is a paragraph. For instance, in some styles of writing, particularly journalistic styles, a paragraph can be just one sentence long. Ultimately, a paragraph is a sentence or group of sentences that support one main idea. In this handout, we will refer to this as the "controlling idea," because it controls what happens in the rest of the paragraph.

✓

✎

🗑

Paragraphs are the building blocks of papers. Many students define paragraphs in terms of length: a paragraph is a group of at least five sentences, a paragraph is half a page long, etc. In reality, though, the unity and coherence of ideas among sentences is what constitutes a paragraph.

✓

✎

🗑

Paragraphs are the building blocks of papers.

✓

✎

🗑

🚀 **GitHub Link:-** <https://github.com/shashikant021/To-Do-React.git>